

Practically Groovy: SwingBuilder and the Twitter API, Part 2

Working with HTTP Basic authentication and the ConfigSlurper

Skill Level: Intermediate

[Scott Davis](#) (scott@thirstyhead.com)

Founder

ThirstyHead.com

17 Nov 2009

In this [Practically Groovy](#) article, Scott Davis continues building the Groovy Twitter client named Gwitter that he began in [Part 1](#). This time, he tackles HTTP Basic authentication and use of Groovy's ConfigSlurper to read in configuration settings.

About this series

Groovy is a modern programming language that runs on the Java™ platform. It offers seamless integration with existing Java code while introducing dramatic new features like closures and metaprogramming. Put simply, Groovy is what the Java language would look like had it been written in the 21st century.

The key to incorporating any new tool into your development toolkit is knowing when to use it and when to leave it in the box. Groovy can be extremely powerful, but only when applied properly to appropriate scenarios. To that end, the *Practically Groovy* series explores the practical uses of Groovy, helping you learn when and how to apply them successfully.

In "[SwingBuilder and the Twitter API, Part 1](#)," I created a Twitter client (named Gwitter) by building out a simple Swing GUI using Groovy's SwingBuilder. I demonstrated the generic Twitter Search API, which conveniently bypasses the need for authentication. But the typical Twitter usage pattern is to get the tweets of the people you follow (*friends* in Twitter-speak) and post your own tweets. For you to

be able to read your friends' tweets, Twitter must first know who you are. So you'll need to be able to *log in* programmatically with Gwitter, using the Twitter API.

In this article, you'll add authentication capability to Gwitter and enable Gwitter to request and parse your friends timeline. Just like last time, you'll start with a simple command-line proof of concept, and then once you are convinced that it works in isolation, you'll integrate it into Gwitter — and make some enhancements to the Gwitter UI. To get the full source code for this article, see [Download](#).

Authentication in Twitter

Twitter supports two methods of authentication: OAuth and HTTP Basic authentication. The Twitter development team states clearly in the documentation that they prefer that you use the OAuth mechanism (see [Resources](#)). But they also admit, "The traditional OAuth flow for desktop clients can be cumbersome." It should come as no surprise, then, that I will take the path of least resistance and show how to use HTTP Basic authentication in Groovy.

Your own Twitter account

You need to supply your own Twitter username and password for the examples in this article to work. If you don't already have an account, visit <https://twitter.com/signup>.

HTTP Basic authentication is popular because it is easy to implement (see [Resources](#)). You simply concatenate the username and the password with a colon, and then Base64-encode the results. It is secure in only the most lenient definition of the word: it keeps plaintext credentials from going out over the wire. But Base64 encoding and decoding is available in every programming language that deals with Web development.

In Groovy, you Base64-encode your Twitter username and password like this:

```
def authString = "username:password".getBytes().encodeBase64().toString()
```

Creating the authentication string is just one tiny piece of the puzzle. In order to read your friends' tweets, you need to make a RESTful request to http://twitter.com/statuses/friends_timeline.atom, passing the Base64-encoded string in the HTTP GET request's `Authorization` header (see [Resources](#)).

Create a file named `friends.groovy`. Add the code in Listing 1 to:

1. Request your friends timeline as an Atom document.

2. Parse it using an XmlSlurper .
3. Print the results to the console.

(For a refresher on parsing XML with a Groovy XmlSlurper, see "[Practically Groovy: Building, parsing, and slurping XML](#)".)

Listing 1. Requesting your friends timeline from Twitter

```
def addr = "http://twitter.com/statuses/friends_timeline.atom"
def authString = "username:password".getBytes().encodeBase64().toString()
def conn = addr.toURL().openConnection()
conn.setRequestProperty("Authorization", "Basic ${authString}")
if(conn.responseCode == 200){
    def feed = new XmlSlurper().parseText(conn.content.text)
    feed.entry.each{entry->
        println entry.author.name
        println entry.title
        println "-"*20
    }
}else{
    println "Something bad happened."
    println "${conn.responseCode}: ${conn.responseMessage}"
}
```

Type `groovy friends` at the command line. The output should look similar to Listing 2. Of course, your output will vary based on the credentials you passed up to Twitter and the friends you follow.

Listing 2. Twitter output from friends.groovy

```
-----
Scott Davis
scottdavis99: @neal4d Is the Bishop's Arms *diagonally* adjacent
to your hotel? Or is it two doors over and one door up?
-----
Neal Ford
neal4d: At the Bishop's Arms, the pub adjacent our hotel, with
a shocking number of single-malt scotches. I need to spend some
quality time here.
-----
```

cURL

cURL is a command-line utility for making HTTP GET, POST, PUT, and DELETE requests. It is standard on UNIX®, Linux®, and Mac OS X systems, and downloadable for Windows® (see [Resources](#)). (For more on using cURL to interact with RESTful Web services, see "[Mastering Grails: RESTful Grails](#)".)

To get your hands on the raw Atom output, you can make a [cURL](#) request from the command line. Listing 3 shows an example:

Listing 3. Using cURL to get the raw Atom from Twitter

```
$ curl -u scottdavis99:password http://twitter.com/statuses/friends_timeline.atom

<?xml version="1.0" encoding="UTF-8"?>
<feed xml:lang="en-US" xmlns="http://www.w3.org/2005/Atom">
  <title>Twitter / scottdavis99 with friends</title>
  <updated>2009-11-03T22:15:45+00:00</updated>

  <entry>
    <title>scottdavis99: @neal4d Is the Bishop's Arms *diagonally*
      adjacent to your hotel? Or is it two doors over and one door up?</title>
    <id>tag:twitter.com,2007:
      http://twitter.com/scottdavis99/statuses/5402041984</id>
    <published>2009-11-03T21:23:43+00:00</published>
    <updated>2009-11-03T21:23:43+00:00</updated>
    <author>
      <name>Scott Davis</name>
      <uri>http://thirstyhead.com</uri>
    </author>
  </entry>

  <entry />
  <entry />
  <entry />
</feed>
```

So, now you're sure that you can make an HTTP GET request using Basic authentication and parse the results. But the hardcoded username and password in the source code should raise a red flag with you. Better to refactor the authentication information into an external properties file.

Reading Java properties with Groovy's ConfigSlurper

One of the most common ways to store external information in a Java application is in a properties file. Properties files are simple key/value pairs stored in plain text. Properties files are flat — they don't offer any sort of nesting or grouping of keys — but you can pseudo-nest your keys by creating dotted names with identical prefixes.

Create a file named `config.properties`, and add the contents of Listing 4:

Listing 4. A simple Java properties file

```
login.username=fred
login.password=wordpass
```

One way to read in properties is with the `java.util.Properties` class. Create a file named `PropertiesTest.groovy` and write the unit test shown in Listing 5:

Listing 5. A simple unit test that reads in a Java properties file

```
class PropertiesTest extends GroovyTestCase{
    void testReadingProperties(){
        Properties properties = new Properties();
        try{
            properties.load(new FileInputStream("config.properties"));
            assertEquals("fred", properties.getProperty("login.username"));
            assertEquals("wordpass", properties.getProperty("login.password"));
        }catch (IOException e) {
            fail(e.getMessage());
        }
    }
}
```

To run the unit test, type `groovy PropertiesTest` at the command line. You should see one dot for each test method, and OK (1 test) at the end of the test run.

You've seen how easy it is to slice thorough XML using an `XmlSlurper`. Groovy offers a similar class — `groovy.util.ConfigSlurper` — for reading in properties files. `ConfigSlurper` makes reading Java properties files trivial.

Add a new test to `PropertiesTest.groovy`, as shown in Listing 6:

Listing 6. Reading Java properties file with a `ConfigSlurper`

```
void testReadingPropertiesWithConfigSlurper(){
    Properties properties = new Properties();
    properties.load(new FileInputStream("config.properties"));
    def config = new ConfigSlurper().parse(properties);
    assertEquals "fred", config.login.username
    assertEquals "wordpass", config.login.password
}
```

As you can see, `ConfigSlurper` lets you use the same dotted notation to walk through Java properties that `XmlSlurper` allows with XML. The `ConfigSlurper` GPath syntax is considerably less noisy than the `properties.getProperty("login.username")` you had to use in [Listing 5's](#) test.

But the benefits of `ConfigSlurper` go beyond syntactic sugar on the reading side. You can use a very Groovy-like set of nested configuration blocks to store your values in the configuration file.

Reading Groovy configuration with a `ConfigSlurper`

A Groovy configuration file is almost a domain-specific language (DSL) for storing your configuration settings. You can group like settings together in a block, and you can nest the blocks to any arbitrary depth. This nesting both reduces repetitive typing and makes it clearer which settings are related.

Create a file named `config.groovy` and add the code in Listing 7:

Listing 7. A Groovy configuration file

```
login{
    username = "fred"
    password = "wordpass"
}
```

Add yet another test to `PropertiesTest.groovy` to read in the Groovy configuration file, as shown in Listing 8:

Listing 8. Reading a Groovy configuration file with `ConfigSlurper`

```
void testConfigSlurper(){
    def config = new ConfigSlurper().parse(new File("config.groovy").text)
    assertEquals "fred", config.login.username
    assertEquals "wordpass", config.login.password
}
```

Because the overloaded `parse` method can accept a simple `String`, it is trivial to mock out the configuration settings in a unit test. Groovy's triple quotes come in handy for this, as shown in Listing 9:

Listing 9. Creating mock settings

```
void testMockConfig(){
    def mockConfig = """
        smtp{
            server = "localhost"
            port = 25
            auth{
                user = "testuser"
                password = "testpass"
            }
        }
    """

    def config = new ConfigSlurper().parse(mockConfig)
    assertEquals "testuser", config.smtp.auth.user
}
```

Using a Groovy configuration file (instead of a Java properties file) in coordination with a `ConfigSlurper` offers one more concrete benefit. You can flip among values based on predefined environments such as `development`, `production`, and `testing`. This is an enormous boon for testability.

Using environments with `ConfigSlurper`

If you create a specially named block — `environments` — in your Groovy

configuration file, you can selectively override configuration settings. If you are a Grails user, you are intimately aware how this works in `grails-app/conf/DataSource.groovy`, as shown in Listing 10:

Listing 10. Grails using a ConfigSlurper to manage its database connections

```
dataSource {
    pooled = true
    driverClassName = "org.hsqldb.jdbcDriver"
    username = "sa"
    password = ""
}

// environment specific settings
environments {
    development {
        dataSource {
            dbCreate = "create-drop" // one of 'create', 'create-drop', 'update'
            url = "jdbc:hsqldb:mem:devDB"
        }
    }
    test {
        dataSource {
            dbCreate = "update"
            url = "jdbc:hsqldb:mem:testDb"
        }
    }
    production {
        dataSource {
            dbCreate = "update"
            url = "jdbc:hsqldb:file:prodDb;shutdown=true"
        }
    }
}
```

In Listing 10, the final `dataSource` config object that the `ConfigSlurper` passes back to you is the combination of the "global" values in the top `dataSource` block and the environment-specific settings in the `development`, `test`, or `production` block. Values that you set in the `environments` block will override the global values.

The outer block must be named `environments`, but you can use any names you'd like for the inner blocks. Add the code in Listing 11 to `config.groovy`, and then test it in `PropertiesTest.groovy`:

Listing 11. Using environments in a Groovy configuration file

```
//config.groovy
login{
    username = "fred"
    password = "wordpass"
}

environments{
    qa{
        login{
            username = "testuser"
        }
    }
}
```

```
        password = "testpass"
    }
}

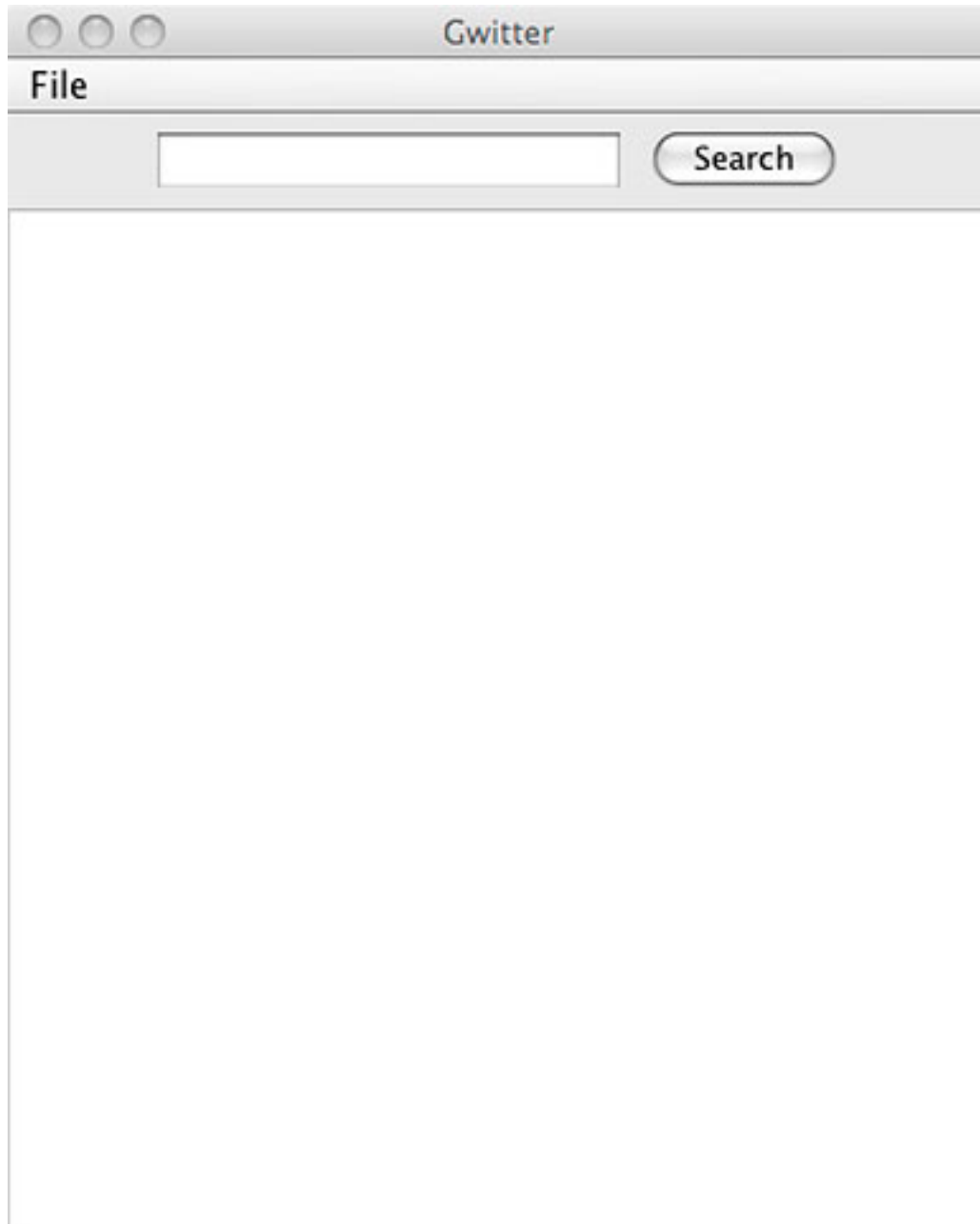
//PropertiesTest.groovy
void testWithEnvironment(){
    def config = new ConfigSlurper("qa").parse(new File("config.groovy").text)
    assertEquals "testuser", config.login.username
    assertEquals "testpass", config.login.password
}
```

Now that you have a bit of ConfigSlurper experience under your belt, you'll combine it with the friends timeline example you ran earlier and add them both to the Gwitter Swing application. The first thing you need to do is switch to a tabbed interface so that you have room to display the new information.

Adding a TabbedPane to Gwitter

At the end of [Part 1](#), you were left with a single-paned application, as shown in Figure 1:

Figure 1. The single-paned Gwitter application



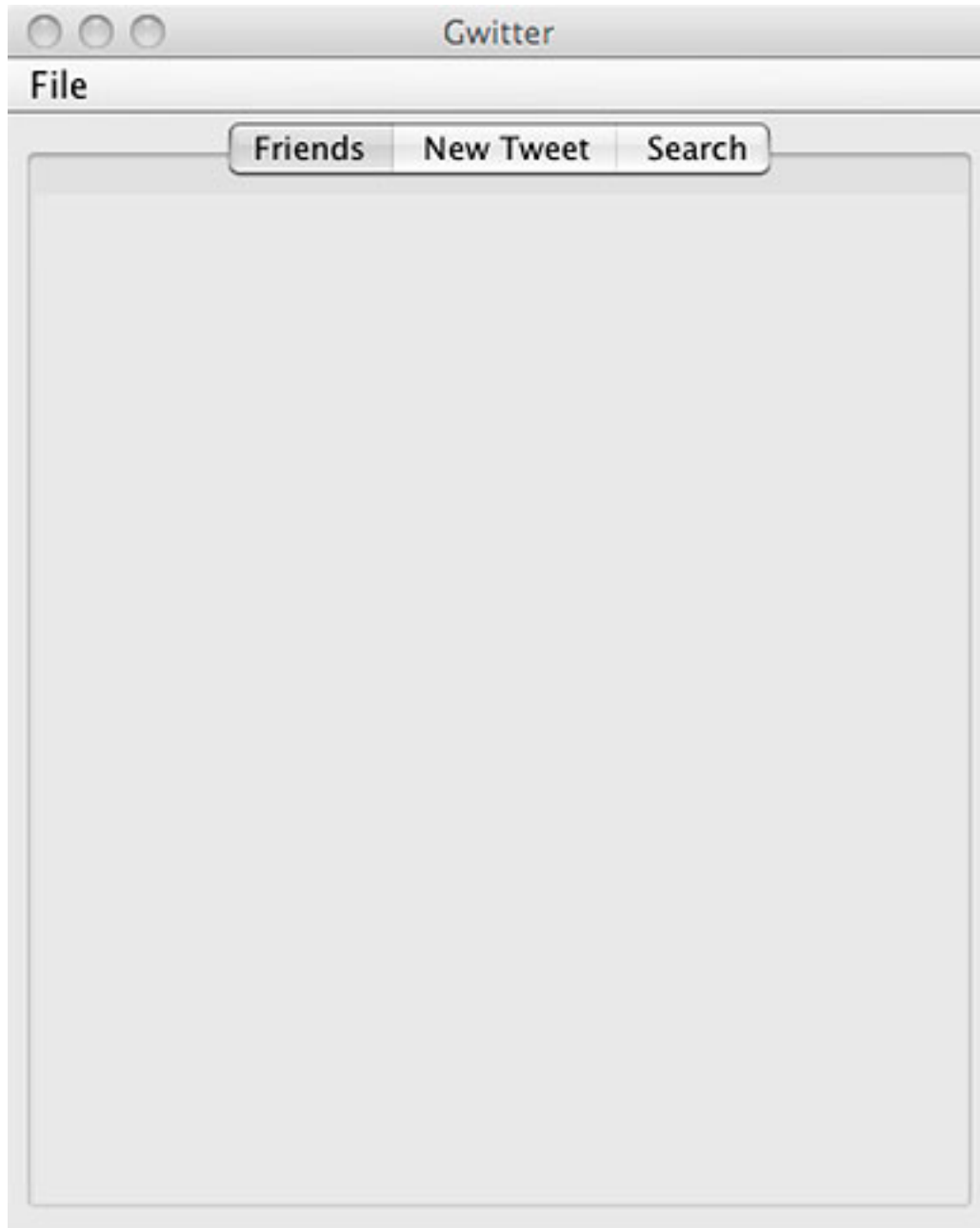
To display the friends timeline, you need to change to a tabbed view. Recall that Groovy's `SwingBuilder` allows you to move away from the typical Java syntax of `JTabbedPane tabpane = new JTabbedPane();` and simply add a nested `tabbedPane` closure to your `SwingBuilder.frame`. (It is strangely reminiscent of a Groovy configuration file and `ConfigBuilder`, isn't it?) Add a new `tabbedPane` to `Gtwitter`, as shown in Listing 12:

Listing 12. Adding a `tabbedPane` to `Gtwitter`

```
swingBuilder.frame(title:"Gtwitter",
                    defaultCloseOperation:JFrame.EXIT_ON_CLOSE,
                    size:[400,500],
                    show:true) {
    customMenuBar()
    tabbedPane{
        panel(title:"Friends")
        panel(title:"New Tweet")
        panel(title:"Search"){
            searchPanel()
            resultsPanel()
        }
    }
}
```

The `title` attribute, not surprisingly, is the label that will be displayed on the tab. Type `groovy Gtwitter` at the command prompt, and you should see your new tabs, as shown in Figure 2:

Figure 2. Gtwitter with a new tabbed interface



I can't emphasize enough that SwingBuilder is simply adding a layer of syntactic sugar over traditional Java Swing. All of your old Java tricks are still applicable in Groovy. For example, rather than instantiating the `JTabbedPane` on one line and calling a setter method like `setTabPlacement()` on a second line, you can do both in a single line, as shown in Listing 13:

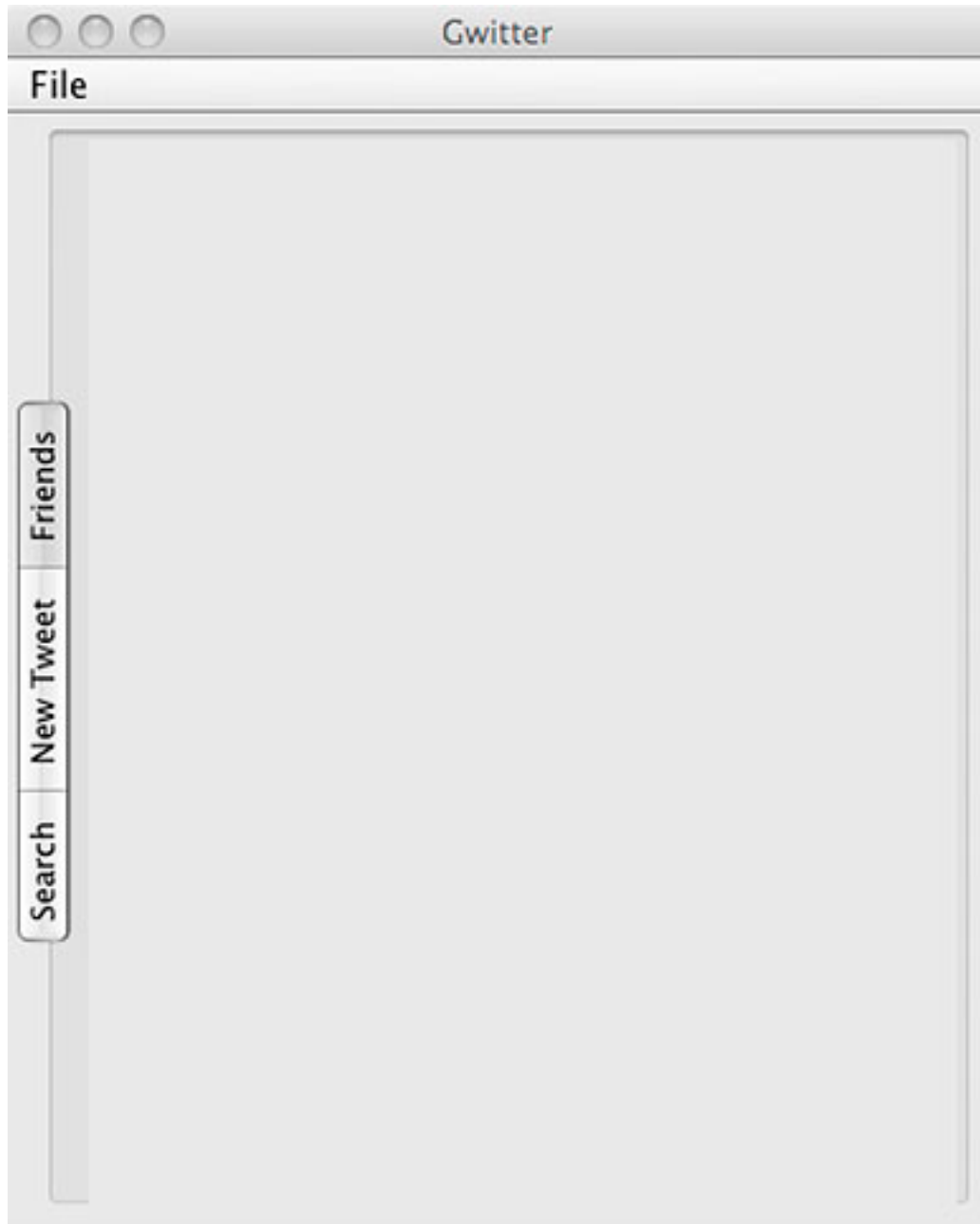
Listing 13. Placing the tabs along the left of the tabbed panel

```
swingBuilder.frame(title:"Gtwitter",
```

```
        defaultCloseOperation: JFrame.EXIT_ON_CLOSE,  
        size:[400,500],  
        show:true) {  
    customMenuBar()  
    tabbedPane(tabPlacement: JTabbedPane.LEFT){  
        panel(title:"Friends")  
        panel(title:"New Tweet")  
        panel(title:"Search"){  
            searchPanel()  
            resultsPanel()  
        }  
    }  
}
```

This, as you might expect, aligns the tabs along the left of the panel instead of the top by default, as shown in Figure 3:

Figure 3. Aligning the tabs along the left of the tabbed pane



After you've run the application again to prove that the left tab placement works as expected, remove the setting to return the tabs to their default placement along the top of the panel.

Populating the Friends tab

Now you'll fill in the empty Friends tab with live data from the Twitter API.

First, create a `gwitterConfig.groovy` file in the same directory as `Gwitter.groovy`. Add

the code shown in Listing 14 (remembering to change the credentials to your Twitter username and password):

Listing 14. The gtwitterConfig.groovy file

```
login{
    username = "username"
    password = "password"
}
```

Next, create a FriendsTimeline.groovy file in the same directory. Add the code shown in Listing 15:

Listing 15. The FriendsTimeline.groovy file

```
class FriendsTimeline{
    static final String addr = "http://twitter.com/statuses/friends_timeline.atom"

    static Object[] refresh(){
        def results = []

        def configFile = new File("gtwitterConfig.groovy")
        if(configFile.exists()){
            def config = new ConfigSlurper().parse(configFile.text)

            //NOTE: this should be a single line in your code
            def authString = "${config.login.username}:${config.login.password}"
                .getBytes().encodeBase64().toString()
            def conn = addr.toURL().openConnection()
            conn.setRequestProperty("Authorization", "Basic ${authString}")
            if(conn.responseCode == 200){
                def feed = new XmlSlurper().parseText(conn.content.text)
                feed.entry.each{entry->
                    def tweet = new Tweet()
                    tweet.author = entry.author.name
                    tweet.published = entry.published
                    tweet.content = entry.title
                    results << tweet
                }
            }else{
                println "Something bad happened."
                println "${conn.responseCode}: ${conn.responseMessage}"
            }
        }else{
            println "Cannot find ${configFile.name}."
        }
        return results as Object[]
    }
}
```

Listing 15 is nearly identical to the code in Search.groovy (see [Part 1](#)). The difference is that in FriendsTimeline.groovy, you are reading in the values from gtwitterConfig.groovy using a ConfigSlurper and using HTTP Basic authentication. In Search.groovy, you made a simple unauthenticated HTTP GET request.

You incorporate the `FriendsTimeline` class into the main `Gtwitter` class in much the same way you incorporated the `Search` class in [Part 1](#). First, add a field for the `friendsList`, as shown in Listing 16:

Listing 16. Adding a `friendsList` field to `Gtwitter`

```
class Gtwitter{
  def searchField
  def resultsList
  def friendsList

  // snip...
}
```

Next, add the panels for a Refresh button and the results of the `FriendsTimeline.refresh()` method call, as shown in Listing 17:

Listing 17. Adding two new panels to `Gtwitter`

```
class Gtwitter{
  def searchField
  def resultsList
  def friendsList

  void show(){
    // snip...

    def friendsRefreshPanel = {
      swingBuilder.panel(constraints: BorderLayout.NORTH){
        button(text:"Refresh", actionPerformed:{
          doOutside{
            friendsList.listData = FriendsTimeline.refresh()
          }
        })
      }
    }

    def friendsPanel = {
      swingBuilder.scrollPane(constraints: BorderLayout.CENTER){
        friendsList = list(fixedCellWidth: 380,
                          fixedCellHeight: 75,
                          cellRenderer:new StripeRenderer())
      }
    }
  }
}
```

Finally, render the `friendsRefreshPanel` and `friendsPanel` in the Friends tab, as shown in Listing 18:

Listing 18. Rendering the two panels in the Friends tab

```
swingBuilder.frame(title:"Gtwitter",
                   defaultCloseOperation:JFrame.EXIT_ON_CLOSE,
                   size:[400,500],
```

```
        show:true) {  
    customMenuBar()  
    tabbedPane{  
        panel(title:"Friends"){  
            friendsRefreshPanel()  
            friendsPanel()  
        }  
        panel(title:"New Tweet")  
        panel(title:"Search"){  
            searchPanel()  
            resultsPanel()  
        }  
    }  
}
```

Type `groovy Gwitter` at the command prompt, and click the Refresh button. Your results should look similar to Figure 4:

Figure 4. Gwitter displaying the friends timeline



Conclusion

In this article, you learned how to make an HTTP GET request using Basic authentication. You also learned how to make the most of Groovy configuration files and the `groovy.util.ConfigSlurper`. And finally, you added a tabbed interface to Gtwitter to display the results of the RESTful Twitter API call to return your friends timeline.

Next time, you'll make an HTTP POST request to add your own tweets. Along the way, you'll also learn about `JTextArea` fields and how to limit their input to a

specific number of characters (oh, say 140?) using a `DocumentSizeFilter`. Until then, I hope that you find plenty of practical uses for Groovy.

Downloads

Description	Name	Size	Download method
Source code	j-pg11179.zip	21KB	HTTP

[Information about download methods](#)

Resources

Learn

- [Groovy](#): Learn more about Groovy at the project Web site.
- [Twitter authentication](#): Read about Twitter's authentication options on the Twitter API Wiki.
- [Basic access authentication](#): This Wikipedia article covers HTTP Basic authentication.
- [Twitter REST API Method: statuses friends_timeline](#): Check out the Twitter API Wiki documentation for the statuses/friends_timeline method.
- [ConfigSlurper](#): Read about ConfigSlurper in the Groovy User Guide.
- [AboutGroovy.com](#): Keep up with the latest Groovy news and article links.
- [Groovy Recipes](#) (Scott Davis, Pragmatic Programmers, 2008): Learn more about Groovy and Grails in Scott Davis's latest book.
- [Mastering Grails](#): Scott Davis's companion series focuses on this Groovy-based platform for Web development.
- Browse the [technology bookstore](#) for books on these and other technical topics.
- [developerWorks Java technology zone](#): Find hundreds of articles about every aspect of Java programming.

Get products and technologies

- [Groovy](#): Download the latest Groovy ZIP file or tarball.
- [cURL](#): Download cURL.

Discuss

- [Groovy mailing list](#): Browse, search, or subscribe to the Groovy mailing list.
- Get involved in the [My developerWorks community](#).

About the author

Scott Davis

Scott Davis is an internationally recognized author, speaker, and software developer. He is the founder of [ThirstyHead.com](#), a Groovy and Grails training company. His books include [Groovy Recipes: Greasing the Wheels of Java](#), [GIS for Web Developers: Adding Where to Your Application](#), [The Google Maps API](#), and [JBoss At Work](#). He writes two ongoing article series for IBM developerWorks: [Mastering Grails](#)

and *Practically Groovy*.

Trademarks

Java and all Java-based trademarks are trademarks of Sun Microsystems, Inc. in the United States, other countries, or both.